Cloud Computing Explained

1. Platform as a Service (PaaS)

Definition

- PaaS is a cloud computing model where a third-party provider gives a complete platform for developing, deploying, and managing applications.
- Developers can focus only on building apps, without worrying about servers, OS, databases, or infrastructure.
- It provides a ready-to-use environment for application development in the cloud.

Key Characteristics of PaaS

- Provides a complete development and deployment environment in the cloud.
- Developers can build anything from simple apps to complex enterprise software.
- Gives tools that let developers create customized solutions using the platform's tools.
- Platforms may be based on:
  - Specific programming languages
  - Application frameworks
  - Other development tools
- Tools and services are fully integrated (IDE, UI tools, security tools, workflow tools).
- Must support UI creation tools such as HTML, JavaScript, and other rich-media technologies.

Interaction & Responsibilities

- Users can interact with the platform: enter data, retrieve data, perform actions, and customize it (if vendor allows).
- **The customer:**
  - Uses the platform
  - Does not manage hardware, servers, OS, or maintenance

- **The vendor:**
  - Handles infrastructure, maintenance, and the software's entire lifecycle

Examples of PaaS

- Google App Engine
  - Developers use Google's APIs.
  - File system and datastore structure are defined by Google.
- Force.com (Salesforce developer platform)
- Google services considered as PaaS:
  - Google Maps
  - Google Earth
  - Gmail
  - Other Google web services

Vendor Lock-In Issue

- Apps built for one PaaS environment generally run only in that specific vendor's platform.
- **Example:** Python app using Google App Engine APIs will work only in Google's environment.

Benefits of PaaS

- Reduced coding time due to built-in components (workflow, security, search, etc.).
- More development capabilities without hiring more team members.
- Remote/distributed team support since everything is cloud-based.
- Develop for multiple platforms (mobile, PC, web).
- Complete app lifecycle support (build → test → deploy → update).
- Cost-effective because teams don't build everything from scratch.
- Helps businesses in:
  - Digital transformation
  - Increased flexibility
  - Productivity
  - Business continuity
- Provides secure data storage, sophisticated tools, and streamlined operations.

2. Software as a Service (SaaS)

Definition

- SaaS delivers software applications over the internet.
- Users access apps through a browser (e.g., email, office tools).
- You pay as you use (subscription or usage-based).
- No installation required.

Characteristics of SaaS

- Software available globally via internet/browser.
- Subscription-based or usage-based billing.
- Vendor manages everything:
  - Hardware
  - Software
  - Updates
  - Maintenance
- Users may have small client-side code, but they don't maintain it.
- Lower costs compared to traditional software.
- Automated updates, patches, and faster rollout.
- Low entry barrier, known recurring cost, easy scalability.
- All users get the same version, ensuring compatibility.
- Supports multiple users with a shared data model (multi-tenancy).
- Sometimes software virtualization is used, but less common.

Examples of SaaS

- Gmail
- Google Calendar
- QuickBooks Online
- Zoho Office Suite

- Online CRM systems
- Online HR systems
- Online Helpdesk systems

Customization in SaaS

- Many SaaS apps cannot be deeply customized; only preferences can be changed.
- But some SaaS apps expose APIs for developers to build:
  - Custom workflows
  - Data schema changes
  - Security model changes
- **Examples:**
  - Salesforce.com
  - Quicken.com

Benefits of SaaS

- No need to update software yourself.
- No licensing or software purchase costs.
- No server cost.
- Pay only for what you use.
- Access data from anywhere, anytime.
- No technical/IT maintenance required.
- No separate budget for security.
- Data is automatically backed up regularly.
- Cloud servers use cluster infrastructure:
  - If one server fails, others continue without interruption.

---

1. Shrink-Wrapped vs Hybrid vs SaaS Licensing

This compares how software is licensed, where it runs, and who manages it.

A. Shrink-Wrapped Software (Traditional Software)

- **Licensing:** You own the software (one-time purchase).
- **Location:** Installed locally on your computer.
- **Management:** Managed by local IT staff.
- **Example:** Microsoft Office installed on your PC.

B. Hybrid Model

- **Licensing:** Subscription (flat monthly/yearly fee).
- **Location:** Accessed using an application (not fully cloud).
- **Management:** Handled by an Application Service Provider (ASP).
- **Example:** Old-style online services where an application connects to a hosted system.

C. SaaS (Software as a Service)

- **Licensing:** Metered subscription (pay for usage).
- **Location:** Fully cloud-based, accessed via browser.
- **Management:** Managed by cloud vendor under a Service Level Agreement (SLA).
- **Example:** Gmail, Salesforce, QuickBooks Online.

---

2. Open SaaS

“Open SaaS” = SaaS software built using open-source technologies.

Why Open SaaS is popular

- Cheaper deployment (no purchase of OS/software).
- Less vendor lock-in (you aren’t stuck with one provider).
- More portability (can move to another cloud easily).
- Open-source stack (LAMP: Linux, Apache, MySQL, Perl/PHP) is widely used and understood.

Leads to:

- Lower development cost
- Higher profitability

- More secure and robust systems

---

3. Open SaaS and SOA (Service-Oriented Architecture)

Many SaaS solutions are component-based, which helps them support mashups.

What is a Mashup?

A web application that combines data/features from multiple sources into one interface.

> ***Example:*** *Showing restaurant locations on Google Maps + adding reviews from Yelp.*

Mashups require 3 components

1. **User Interface:**
   - Built using HTML, XHTML, CSS, JavaScript, Ajax.
2. **Web Services (APIs):**
   - Accessible through protocols like: SOAP, REST, XML/HTTP, XML/RPC, JSON/RPC
3. **Data Formats:**
   - XML, JSON, KML (Keyhole Markup Language used in Google Earth/Maps)

Mashups in the cloud industry

- SaaS makes mashups easy to build.
- Open Mashup Alliance (OMA) supports these technologies.
- They created EMML (Enterprise Mashup Markup Language), a DSL (domain-specific language).
- Usage of mashups expected to grow 10x in a few years.

SaaS Aggregators

- Gartner predicted 25% of all software would be SaaS.
- An aggregator combines SaaS apps from many vendors and presents them as a single unified solution.

---

4. Salesforce.com and CRM SaaS

Salesforce is the most famous SaaS example.

Features offered in Salesforce CRM:

- Sales
- Customer service
- Support
- Marketing
- Content management
- Analytics
- Collaboration via Chatter

History

- Founded in 1999 by former Oracle employees.
- Early leader in SaaS technologies.

Salesforce as PaaS: Force.com

Salesforce extended its platform so developers can build apps on top of it.

Force.com includes:

- Programming language Apex (Java-like language)
- UI creation using XML syntax, HTML, Ajax, Flex

Today, nearly 1000+ applications exist on the platform.

---

1. What Are Open Standards?

- Open standards define how data is formatted, stored, and exchanged.
- They allow different systems, companies, and applications to communicate smoothly.
- They are created through a public, collaborative, consensus-based process (not owned by one company).

- They promote:
  - Compatibility
  - Interoperability
  - Innovation
  - No vendor lock-in

2. Open Standards in Cloud Computing

- Cloud environments depend heavily on interoperability (systems working together).
- Interoperability has two types:
  - **Horizontal** → Between two different applications
  - **Vertical** → Among components within the same application

3. Why Cloud Needs Open Standards

- Cloud brings massive scalability and new deployment models.
- Without open standards, vendors create proprietary “stovepipe clouds”:
  - These cause vendor lock-in
  - Clients cannot switch providers easily
- So, the industry pushes for standard-based clouds to avoid lock-in.

4. Architectural Standards Used in Cloud Computing

Cloud computing relies on many open architectural standards:

- Platform virtualization
- Service-Oriented Architecture (SOA)
- Web application frameworks
- Open-source software deployment
- Standardized web services (REST, SOAP, etc.)
- Autonomic systems (self-managing systems)
- Grid computing

These standards support cloud business models like:

- SaaS
- Web 2.0 applications
- Utility computing

They ensure data portability and universal accessibility.

5. The Race for Open Cloud Platforms

Cloud companies compete to build open cloud platforms that can rival proprietary ones like:

- Microsoft Azure
- VMware vSphere

6. OpenStack: An Important Open-Source Cloud Project

- Started by Rackspace (IaaS provider) in July 2010.
- Built using Rackspace's Cloud Files and Cloud Servers technology.
- NASA contributed technology from its Nebula Cloud Platform.
- Released under the Apache 2.0 open-source license.
- Founding members include AMD, Citrix, Dell, Intel, NTT Data, and others.
- OpenStack's home page: openstack.org

7. Eucalyptus: Another Open Cloud Platform

- **Eucalyptus** = Elastic Utility Computing Architecture for Linking Your Programs to Useful Systems
- A Linux-based platform for building IaaS cloud systems on clusters.
- Can connect to Amazon AWS services:
  - EC2 (compute)
  - S3 (storage)
- Supports many virtualization technologies:
  - VMware
  - Xen
  - KVM

- Supported by major Linux vendors.
- Created at University of California, Santa Barbara.
- Eucalyptus Systems (company) formed in 2009 to commercialize it.

8. Other Open Cloud Initiatives

- Many other open-source cloud projects exist globally.
- Many research groups are working on open cloud standards.
- The **IEEE Technical Committee on Services Computing**:
  - Runs the CLOUD conference.
  - Manages working groups and publications in cloud standards.

9. OpenStack Modules (Deliverables)

OpenStack initially released two major components:

1. OpenStack Compute

- Automatically creates large pools of virtual private servers.
- Works on common, industry-standard hardware.

2. OpenStack Storage

- Provides redundant, object-based storage.
- Runs on clusters of inexpensive (commodity) servers and storage devices.

These came from:

- Rackspace Cloud Files
- NASA Nebula technology

---

1. Introduction to IaaS

Cloud computing can be viewed in four layers:

1. Application Layer → SaaS

2. Platform Layer → PaaS
3. Infrastructure Layer → IaaS
4. Hardware Layer → Physical resources

What is IaaS?

- IaaS provides virtualized hardware over the cloud.
- It functions like utility computing → you draw resources only when needed.
- You pay only for what you use (pay-as-you-go).
- It enables massive scalability, turning small companies into large operations quickly.
- It is considered a disruptive technology because of its rapid deployment capabilities.

2. How IaaS Works

Vendor Responsibilities

The IaaS provider owns and manages:

- Physical servers
- Storage devices
- Network infrastructure
- Data centers
- Virtualization layer

Developer Responsibilities

Developers create:

- Virtual servers (instances)
- Virtual storage
- Virtual networks
- Applications deployed on the above

> ***Key Idea:*** *IaaS creates virtual hardware for developers. When you request virtual resources, those requests get mapped to physical resources in the provider's data*

> *center.*

3. Workloads in IaaS

The smallest unit in IaaS is called a **workload**.

What is a workload?

- A workload simulates the performance of a real physical server.
- A workload is measured by:
  - TPM (Transactions Per Minute)
  - Disk IOPS (Input/Output operations per second)
  - RAM usage
  - Network throughput & latency

Server Instances

In IaaS, instead of running apps on dedicated physical servers, you reserve a virtual machine instance sized for your workload:

- Small
- Medium
- Large

The system allocates: CPU, RAM, Storage, Network capacity from a pool of virtualized hardware.

4. Logical Layers in IaaS

IaaS separates the system into logical units, not physical machines.

The three logical layers:

1. **LUN (Logical Unit Number)**
   - Logical storage containers
   - Often mapped to RAID storage
2. **Cloud Interconnect Layer**

- A virtual network
- Uses IP addresses from the IaaS provider's IP pool

3. **Virtual Application Software Layer**
   - Software running on the virtual machines

5. Networking Behavior of IaaS

Amazon EC2 networking

- AWS does not support Layer 2 networking (broadcast/multicast).
- This restricts VLANs and certain protocols.

Other providers

- Some providers support Layer 2 through tunneling, enabling:
  - VLANs
  - Private networks
  - More realistic on-premises setup
- **Examples:** Cloudscaling.com, VMware Cloud

6. Example: AWS IaaS

Amazon Web Services is the classic example of an IaaS system.

AWS offers:

- Virtual servers (EC2)
- Virtual storage (S3, EBS)
- Databases
- Messaging/notification systems
- System images (AMIs) for different OS and apps

EC2 servers run on Xen virtualization and are divided into logical compute units.

7. E-Commerce System Example

A typical cloud-hosted e-commerce system includes:

- Web server
- Application server
- File server
- Database
- Transaction engine

Each of these creates multiple workloads: Database queries, Business logic processing, Serving web pages.

8. Pods, Aggregation, and Silos

Pods

- A **Pod** = group of users handled by a particular server instance.
- Pods are managed using a Cloud Control System (CCS).
- In AWS, the CCS is AWS Management Console.
- When an instance reaches its size limit: You must clone the instance to serve more users.

Availability Zones

- Pods are grouped inside **Availability Zones (AZs)**.
- Each AZ is a separate fault domain.
- **Examples of AWS regions and AZs:** Northern California, Northern Virginia, Ireland, Singapore
- Failover between zones gives high availability.

Silos

- A **silo** is a cloud environment that is isolated from others.
- **Characteristics:**
  - Acts like a sealed “compute island”
  - Private virtual networks often result in silos
  - Limits interoperability
  - Caused by architecture or vendor restrictions

- Common in PaaS platforms like: Force.com, QuickBase

Pros and Cons of Silos

- **Advantages:**
  - Highly secure
  - Good for private ecosystems
- **Disadvantages:**
  - Not flexible
  - Hard to integrate with other systems
  - Vendor lock-in

---

1. What is IDaaS?

- **IDaaS** = cloud-based identity and authentication services that enterprises subscribe to.
- It's part of the familiar "X-as-a-Service" model: a feature is delivered remotely by a third party instead of being built/managed on-site.
- **Analogy:** local email (Outlook/Thunderbird) vs cloud email (Gmail). Likewise, identity & security can be provided as a service.
- **Goal:** Verify users are who they claim to be and grant the right access to apps/files/resources at the right time.
- **Why cloud?** On-prem teams must constantly adapt (e.g., BYOD phone changes). A centralized cloud service run by identity experts solves common problems once and scales to many customers.

2. IDaaS isn't new in spirit

- Many network services are "identity" flavored. Example: **DNS (Domain Name System)** is effectively an Internet-wide identity/authorization/lookup service for domains (.com, .org, …).
- DNS links a domain's identity (name/IP) to metadata (owner, records).

3. Digital identity: what it means

- **Identity** = set of attributes/traits that make something recognizable.

- **Digital identity** = attributes + metadata + relationships with other objects that make it uniquely identifiable in context.
- Database analogy: entities/relationships (tables/rows/columns, ORM).
- Profiles (e.g., Facebook) are an extension of digital identity concepts.
- Identity can include (for a person):
  - **Things you are:** age, gender, appearance, biometrics.
  - **Things you know:** SSN/PINs, schooling, biographical data.
  - **Things you have:** bank account access, security keys, fingerprints/retina pattern.
  - **Things you relate to:** family/friends, licenses, beliefs/activities/habits, accounts (e.g., Google ID).

4. Authentication methods

- **Single-factor:** name + password (things you know).
- **Two-factor:** password + token (hardware/OTP) (something you have).
- **Multi-factor:** add biometrics (fingerprint/retina) (something you are).
- MFA often uses an external trust service → a typical cloud IDaaS workload.
- Other digital identities: users, machines, devices.
  - User/machine accounts live in domain directories and federated stores.
  - Network interfaces have MAC (EHA) addresses—key to being found on networks.

5. Example: Windows Product Activation (how identity profiles are built)

- Creates a hardware/software identity profile for your system. Data captured includes:
  - 25-char product key & product ID, GUID
  - PC manufacturer, CPU type/serial, BIOS checksum
  - Network adapter & MAC, display adapter
  - SCSI/IDE adapters, RAM amount
  - Hard drive & volume serial, optical drive
  - Region/language/user locale
- These are weighted into a code stored in a registration DB.

- Change enough factors (NIC/CPU/GPU/RAM/disk) → re-activation prompt.
- (These are proprietary cloud applications for identity/validation.)

6. Why IDaaS matters on networks

- Networks constantly validate identity: hundreds of messages per minute, every Ethernet packet carries identifying headers.
- Identity protection is complex and costly; experts are scarce → managed/hosted IDaaS can be the most valuable and cost-effective subscription you buy.

7. What services can IDaaS include?

- Authentication (identity verification)
- Directory services
- Federated identity
- Identity governance
- Identity & profile management
- Policies, roles, enforcement
- Provisioning (external policy administration)
- Registration
- Risk & event monitoring / audits
- Single Sign-On (SSO) / pass-through auth

Sharing identity attributes may be regulated; expose only what's justified—this defines the identity relationship.

8. Codes of conduct (what good IDaaS must follow)

- User consent & control over identity use.
- Minimal disclosure: share the least needed data.
- Justifiable access: only trusted parties with a valid reason.
- Directional exposure:
  - Public entities → bi-directional discoverable IDs.

  - Private entities → uni-directional identifiers (protects privacy).
- Interoperability with other providers.
- Unambiguous human identification while resisting identity attacks.
- Consistency of service across contexts and tech.

9. IDaaS interoperability standards (to plug into apps easily)

Cloud IDaaS relies on open/industry standards so different systems work together:

- **User-centric authentication / Information cards:**
  - OpenID, CardSpace.
- **Authorization policies:**
  - **XACML** language to write/enforce distributed authorization rules.
  - Works with SAML: SAML asks for authz → XACML evaluates policy → allow/deny.
- **Provisioning (join/move/leave accounts):**
  - **SPML** (OASIS PSTC) request/response for service provisioning; SOA-aligned.
- **Auditing:**
  - **XDAS** (Distributed Audit Service) for accountability & detecting policy violations.
- **Identity Governance Framework (IGF) – Liberty Alliance**
  - Focus: exchange & control of identity info using WS-Trust, ID-WSF, SAML, LDAP.
  - Released CARML (Client Attribute Requirements Markup Language) + IGF Privacy Constraints.
  - Basis for Project Aristotle (open-source API for identity interchange).

10. Big picture: Why choose IDaaS?

- Centralized, expert-run identity services adapt quickly (e.g., BYOD changes).
- Reduce internal complexity/cost; improve security posture.
- Built on interoperable standards so identity can flow safely across apps, orgs, and clouds—with consent, minimal disclosure, and auditability.

---

1. User Authentication in Cloud Systems

User authentication means confirming who the user is.

Cloud systems often use open standards, so different platforms can verify identity securely.

The main technologies discussed are:

- OpenID
- CardSpace
- XACML
- SAML
- SPML

2. OpenID – A User Authentication Standard

What OpenID is:

- An industry standard for authenticating end users.
- Stores a user's digital identity in a common format.
- Identity is stored at an OpenID service provider.
- The provider translates it into a unique URL/XRI identifier.

Identifier formats:

OpenID identifiers can be:

- URL (example: username.wordpress.com)
- XRI (Extensible Resource Identifier)

Any app that supports OpenID can authenticate the user using that provider.

Who are OpenID identity providers?

Large cloud companies support OpenID, including:

- AOL, Google, Facebook, IBM, Microsoft, Yahoo!, PayPal, MySpace, VeriSign, LiveJournal, Ustream, Orange, WordPress, Blogger, etc.

Authentication methods OpenID can use:

OpenID authentication may require:

- Password
- Hardware keys
- Smart cards
- Tokens
- Biometrics

After logging in:

- Once you log in to a trusted OpenID provider, you get access to multiple OpenID-enabled websites.
- The website you visit acts as the: **Relying Party**, which asks the OpenID provider to confirm your identity.

3. CardSpace – Microsoft Authentication System

What is CardSpace?

- A Microsoft identity-management client.
- Built into Windows and based on OASIS Web Services standards:
  - WS-Trust
  - WS-Security
  - WS-SecurityPolicy
  - WS-MetadataExchange

Where it works:

- Included in .NET Framework 3.0+
- Runs on Windows XP, Server 2003, Vista, Windows 7

How it works:

- A website requests an Information Card using an HTML `<OBJECT>` tag.
- CardSpace uses a Security Token Service (STS).
- STS generates an encrypted and digitally signed token for authentication.

Can be used for:

- Signing in to cloud apps
- OpenID-enabled services
- Windows Live ID accounts

CardSpace is essentially an alternative to OpenID and SAML for cloud authentication.

4. Authorization Markup Languages (XML-Based)

In cloud services, identity requests and replies use XML because:

- XML is text-based
- Self-describing (contains schemas)
- Easily exchanged over web protocols

Two important XML standards are:

(A) XACML – eXtensible Access Control Markup Language

- An OASIS standard for defining authorization policies.
- A policy describes:
  - **Subject** (who is requesting)
  - **Action** (what they want to do)
  - **Resource** (what they want to access)
  - **Environment** (conditions)
- All described in XML.
- **Example:** A user (subject) wants to read (action) a given file (resource) if certain conditions are met.

(B) SAML – Security Assertion Markup Language

- Used for authentication + authorization assertions.
- A SAML assertion states: “Assertion X created at Time T by User U about Subject S is true when Conditions C are true.”

SAML supports 3 types of queries:

1. Authentication query
2. Attribute query
3. Authorization decision query

Technical details:

- Uses SOAP binding → embedded in SOAP inside an HTTP message.
- Enables Web Browser Single Sign-On (SSO).

How SAML SSO works:

1. User tries to access a service provider (website).
2. The service provider sends an authentication request to the SAML identity provider.
3. User gets authenticated and returned with an assertion.
4. Browser gets access without re-entering credentials.

5. SPML – Service Provisioning Markup Language

What SPML is:

- An OASIS standard for automating **provisioning**.
- Provisioning = preparing a resource so it is ready to use.

Examples:

Setting up:

- A VPN
- A phone line
- A user account
- A VM instance

Components Involved:

- **Requesting Authority (RA)** → Client
- **Provisioning Service Point (PSP)** → Cloud service handling the request

- **Provisioning Service Target (PST)** → The system where provisioning is applied

Why SPML is important:

It makes cloud systems:

- automated
- consistent
- efficient
- reliable
- able to enforce access rights

Without SPML, cloud provisioning is slow and error-prone.

6. Benefits of IDaaS

Main advantage: Cost Savings

Local identity systems (like Active Directory) are expensive because you must:

- Maintain servers
- Purchase and upgrade software
- Back up data
- Pay hosting fees
- Manage network security
- Set up VPNs
- Perform continuous admin work

With IDaaS:

- All you pay is a subscription fee
- Identity experts manage everything

Other major benefits:

- Improved cybersecurity
- Faster logins

- Reduced password resets
- Consistent and secure login experience (airport Wi-Fi or office desk)
- Avoids breaches that could destroy a business

---

1. What is Compliance as a Service (CaaS)?

- **CaaS** = Compliance as a Service
- It is a cloud-based service model where organizations outsource compliance-related tasks to a third-party provider.

Purpose:

- Helps companies meet government, legal, regulatory, and industry compliance requirements.
- Common in healthcare, finance, banking, insurance, and government—industries with strict rules.

A CaaS provider handles:

- Implementation of compliance controls
- Management of compliance processes
- Continuous monitoring
- Auditing
- Reporting
- Maintaining compliance over time

2. Why Compliance is Difficult in Cloud Computing

The challenge:

- Cloud computing spans multiple countries.
- This creates legal conflicts:
  - The user's country laws
  - The data processing location's laws

  - The cloud provider's country laws
- These laws often do not match, making compliance complicated.

Compliance is NOT simple:

- Not just about giving someone an access token or login
- Involves privacy, policy, data governance, legal rules, security controls, audits, incident handling, etc.
- Compliance requires deep expertise.

3. Why CaaS Is Rare Today

- Although the term appears in discussions, few universal CaaS products exist.
- **Reason:** A compliance service must function as a trusted third party, similar to a man-in-the-middle, which requires high trust and security.

For CaaS to work, it must:

- Be designed as its own layer in a Service-Oriented Architecture (SOA).

CaaS must be capable of:

- Managing cloud relationships
- Understanding and enforcing security policies
- Handling sensitive data & privacy policies
- Being aware of geographical laws
- Providing incident response
- Performing archiving and logging
- Supporting audits and queries
- Providing all of this through a Service Level Agreement (SLA)

If done right, CaaS becomes a very valuable service.

4. Vertical Clouds for CaaS

To support industry-specific compliance, some companies are creating **vertical clouds**—clouds specializing in one market.

Examples of vertical clouds offering CaaS capabilities:

Industry	Example Provider	Note
Medical/Healthcare	athenahealth	Handles medical compliance
Banking	bankserv	Banking compliance
Merchant / Credit Card	ClearPoint	PCI Compliance-as-a-Service
Government	FedCloud	Gov cloud compliance
PCI-Compliant Cloud	Rackspace PCI Compliant Cloud	Another PCI DSS CaaS

Private Cloud CaaS

- Many CaaS solutions exist inside private clouds.
- **Why?** Because the company:
  - Controls the data
  - Controls security
  - Can fully audit transactions
  - Meets compliance rules more easily

5. Why CaaS Can Be Extremely Valuable

Benefits & Potential:

- Helps companies measure compliance risk
- Could guarantee / indemnify customers against compliance failures
- Ensures processes meet industry laws
- Could be extended to future applications, for example:

- Ensuring emails meet standards
- Could be used by national postal systems
- Potentially helping reduce or eliminate spam

A well-built CaaS can transform how compliance and security are handled in cloud systems.

---

1. Cloud Types Overview

Cloud computing is classified using two different sets of models:

A. Deployment Models (Where and how cloud infrastructure is located/managed)

- Shows where the cloud is hosted
- Shows who owns/manages the infrastructure

B. Service Models (What type of service the cloud provides)

- Defines what cloud services you get (IaaS / PaaS / SaaS)

These two are the foundation of all cloud classifications.

2. NIST Cloud Model

The U.S. National Institute of Standards and Technology (NIST) created the most widely accepted definitions for cloud computing.

Why NIST matters:

- The U.S. government is a huge consumer of cloud services.
- They needed clear definitions for:
  - procurement
  - security
  - standardization
  - interoperability

NIST separates cloud computing into:

- Service Models (IaaS / PaaS / SaaS)
- Deployment Models (Private, Public, Hybrid, Community)

3. Important Notes About the NIST Model

Early NIST versions:

- Did not require virtualization
- Did not require multi-tenancy

Latest NIST definition:

- Cloud must use virtualization
- Cloud must support multi-tenancy (multiple customers sharing the same infrastructure)

What NIST does NOT cover:

NIST does not describe intermediary cloud services such as:

- Service brokers
- Transaction brokers
- Provisioning tools
- Integration services
- Interoperability services

These are part of cloud architecture but not defined in the NIST model.

4. Cloud Cube Model (Jericho Forum — The Open Group)

The Jericho Forum created the **Cloud Cube Model** to help organizations understand how secure collaboration works in cloud environments.

Cloud Cube Model shows:

- Where data lives
- Who owns the tech
- How security boundaries shift

- Who provides the service

Purpose:

To classify clouds based on four dimensions and show that 🔥 **traditional firewall boundaries no longer define security in cloud computing.**

5. The Four Dimensions of the Cloud Cube Model

The Cloud Cube Model classifies clouds based on 4 dimensions:

1. Physical Location of Data

- **Internal (I) vs External (E)**
- **Internal (I)** → Data stored inside the organization's boundary
- **External (E)** → Data stored outside the organization
- **Meaning:** Where the data physically lives.

2. Ownership

- **Proprietary (P) vs Open (O)**
- **Proprietary (P)** → Owned by one vendor, less portable, more lock-in
- **Open (O)** → Open standards, easy interoperability, less lock-in
- **Meaning:** Who owns the technology and how easy it is to move data/applications.

3. Security Boundary

- **Perimeterised (Per) vs De-perimeterised (D-p)**
- **Perimeterised (Per)** → Inside firewall, within traditional security boundaries
- **De-perimeterised (D-p)** → Outside the firewall, security handled by cloud controls
- **Meaning:** Whether the cloud is inside or outside your security perimeter.

4. Sourcing

- **Insourced vs Outsourced**
- **Insourced** → Service provided by the customer internally

- **Outsourced** → Service provided by an external cloud vendor
- **Meaning:** Who delivers the service.

6. How Cloud Cube Model Combinations Work

Using the first 3 dimensions, we get 8 possible combinations:

Example blocks:

- Per-Internal-Proprietary (Per-IP)
- Per-Internal-Open (Per-IO)
- Per-External-Proprietary (Per-EP)
- Per-External-Open (Per-EO)
- D-p Internal Proprietary (D-p IP)
- D-p IO, D-p EP, D-p EO

The 4th dimension (sourcing) applies to each of them, creating more granularity.

7. What the Cloud Cube Model Shows

The model demonstrates:

- Traditional idea of "inside firewall = safe" is outdated.
- Cloud computing moves the security boundary.
- Organizations must design security based on:
  - Location
  - Ownership
  - Perimeter
  - Sourcing

...Not just a firewall.

---

1. Cloud Deployment Models (NIST Definition)

Deployment models describe where the cloud infrastructure is located and who controls it. NIST defines four major deployment models:

1) Public Cloud

- **Definition:** Cloud infrastructure is available for general public use or large industry groups. Owned and operated by third-party cloud providers.
- **Key Characteristics:**
  - Resources shared among multiple customers
  - Multi-tenancy: data of many organizations stored together
  - Pay-per-use model
  - Highly scalable and flexible
  - Least expensive
  - Accessible over the internet

2) Private Cloud

- **Definition:** Cloud infrastructure used exclusively by one organization.
- **Key Characteristics:**
  - Resources shared only within one organization
  - Greater control, predictability, consistency
  - Can be managed by the organization or a third party
  - Can be on-premises or off-premises
  - More expensive
  - Restricted access (internal use only)

3) Hybrid Cloud

- **Definition:** A combination of public + private + community clouds that remain unique but work together.
- **Key Characteristics:**
  - Allows data/applications to move between clouds
  - Supports standardized or proprietary access
  - Offers application portability
  - Flexible and scalable
  - Cost depends on mix of cloud types
  - Operated by both private and public providers

4) Community Cloud

- **Definition:** Cloud infrastructure designed for a group of organizations with common goals or compliance needs.
- **Key Characteristics:**
  - Shared by multiple organizations with similar missions or policies
  - Examples: government agencies, banks, health institutions
  - Managed by the organizations or a third party
  - Apps.gov (US Government Cloud) is a classic example

2. Deployment Model Comparison

Factors	Public Cloud	Private Cloud	Hybrid Cloud
Resources	Shared among multiple customers	Shared with a single organization	Combination of public + private
Tenancy	Multi-organization	Single organization	Data shared across both clouds
Pay Model	Pay-as-you-use	Various pricing models	Pay-as-you-go + fixed pricing + subscriptions
Operated by	Third-party provider	Specific organization	Both
Scalability	High scalability/flexibility	High consistency/predictability	High scalability via mixed approach
Cost	Less expensive	More expensive	Depends on requirements
Availability	Available to the public	Restricted to one org	Combination

3. Cloud Service Models (SPI Model)

Cloud services follow the XaaS (Anything as a Service) pattern. The three universally accepted models are:

1) Infrastructure as a Service (IaaS)

- **What it provides:** Virtual machines, Virtual storage, Virtual networks, Other hardware resources.
- **Who manages what?**
  - **Provider:** manages infrastructure
  - **Client:** installs OS, applications, manages data, user interaction
- **Examples:** Amazon EC2, Rackspace Cloud, Linode, Eucalyptus, Terremark

2) Platform as a Service (PaaS)

- **What it provides:** Infrastructure + operating system, Middleware (DB, runtime, APIs), Development frameworks, Transaction services, Tools for deployment.
- **Who manages what?**
  - **Provider:** infrastructure + OS + platform software
  - **Client:** only the applications they develop or deploy
- **Examples:** Google App Engine, Force.com, Windows Azure Platform, GoGrid CloudCenter

3) Software as a Service (SaaS)

- **What it provides:** Complete software applications hosted in the cloud. Customers only use the software via a browser.
- **Who manages what?**
  - **Provider:** entire stack (infrastructure → OS → app)
  - **Client:** only data + user interaction
- **Examples:** QuickBooks Online, Google Apps (Gmail, Docs), Salesforce.com, Oracle OnDemand, SQL Azure

4. Relationship Between IaaS → PaaS → SaaS (Cloud Stack)

Cloud models form a layered stack:

```
SaaS (Highest Integration – Least Control)
↑
PaaS
↑
IaaS (Lowest Integration – Most Control)
```

- IaaS provides the base hardware infrastructure
- PaaS builds on top of IaaS (adds OS, middleware, frameworks)
- SaaS sits on top of PaaS (complete applications)

The stack is sometimes called the Cloud Reference Model.

5. Example: Evolution of Cloud Service Providers

Salesforce.com

- Started as SaaS (CRM application).
- Later added Force.com → PaaS platform via APIs.

Microsoft Azure

- Started as PaaS for .NET apps.
- Later added IaaS (VMs, storage).
- Now includes SaaS apps (Microsoft 365, SQL Azure).

This shows cloud service models overlap and evolve.

---

1. Virtualized Cloud Model

A virtualized cloud has these layers:

1. **Applications:** Software used by end users.
2. **Operating System:** Manages hardware and software resources.
3. **Virtual Machine (VM):** A software-based computer created by virtualization.
4. **Virtual Infrastructure:** Combination of virtual servers, virtual storage, virtual networks.
5. **Network & Storage:** Physical cables, routers, switches, disks on which virtualization is built.

> ***Conclusion:*** *Cloud = Virtualization + Network + Storage + Applications delivered as services.*

2. Features of Cloud Computing

- **Resource Pooling** – Resources shared across multiple customers.
- **On-Demand Self-Service** – Users can allocate resources whenever they need.
- **Easy Maintenance** – Automatic updates & management.
- **Large Network Access** – Available through the internet from anywhere.
- **High Availability** – Services available 24/7.
- **Automatic Systems** – Auto-scaling, auto-healing.
- **Economical** – Pay only for what you use.
- **Security** – Built-in authentication, encryption.
- **Pay-As-You-Go** – Metered billing.
- **Measured Service** – Usage is monitored and billed accordingly.

3. Cloud Computing – Challenges

%	Issue
97%	Many organizations use multiple clouds (public, private, hybrid) → complexity
65%	Difficulty defining cloud strategy
10%	Fear of reduced investment
53%	Unauthorized access
44%	Hijacking of accounts
49%	Insecure API/Interfaces
33%	External data sharing risks

4. What is Cloud Computing? (Definition)

Cloud computing is:

- A model that provides on-demand access to shared computing resources over the internet.
- Users can access servers, storage, databases, software, and applications.
- Users pay only for what they use.
- Cloud hides infrastructure complexity with simple interfaces (GUI/API).

5. Characteristics of Cloud Computing

- On-demand self-service
- Scalability
- Flexibility
- Broad network access
- Reliability
- Resource pooling
- Security
- Collaboration
- Rapid elasticity
- Automation
- Real-time analytics
- Measured service
- Compliance & Governance
- Multi-tenancy
- High-performance computing

6. Applications of Cloud Computing

- Data storage & management
- Web and mobile apps
- Big data analytics

- Internet of Things (IoT)
- AI & Machine Learning
- Online learning
- Social media tools
- Gaming & Virtual Reality
- Cybersecurity
- CRM, SCM, FM, Accounting
- Healthcare & medical research

7. Cloud Essentials

(A) Abstraction

Cloud abstracts (hides) the underlying complexity:

- Users don't know where data is stored
- Developers don't need to manage infrastructure
- Admin responsibilities are outsourced
- Access is universal from anywhere

(B) Virtualization

- Resources are pooled and shared
- Storage, compute, network can be provisioned dynamically
- Supports multi-tenancy
- Cost is based on usage
- Enables rapid scaling, agility

8. How Cloud Changed System Deployment (Examples)

- **Google:** Built huge global datacenters to run search engine. Earned advertising revenue. Provides free apps (Docs, Gmail, etc.)
- **Microsoft Azure Platform:** Allows .NET applications to run on cloud. Supports hybrid setup (local + cloud).

- **Amazon Web Services (AWS):** Most successful IaaS provider. Allows customers to rent virtual servers (EC2), databases, storage.

9. Evolution of Cloud Computing

Cloud evolved from:

- Autonomic computing
- Client-server model
- Grid computing
- Mainframe computing
- Utility computing
- Peer-to-peer computing
- Virtualization

Goal of Cloud Computing:

- Use all these technologies without needing deep knowledge
- Reduce cost & IT complexity
- Let users focus on business

10. Deployment Models (NIST)

1) Private Cloud

- Owned by enterprise
- On-premise or off-premise
- High control and security

2) Public Cloud

- Sold to public
- Massive scale
- Cheaper

3) Hybrid Cloud

- Combination of two or more clouds
- Standardized or proprietary access
- Movement of data/apps allowed

4) Community Cloud

- Shared by multiple organizations with common needs
- Example: US Govt Apps.gov

11. Deployment Models Table Summary

Factor	Public Cloud	Private Cloud	Hybrid Cloud
Resources	Shared with many	Single org	Combination
Tenancy	Multi-org	Single org	Mixed
Pay Model	Pay-as-you-use	Many pricing types	Combination
Operated by	Third-party	Organization	Both
Scalability	High	Consistent & predictable	High
Cost	Low	High	Depends
Availability	Public	Restricted	Both

12. Service Models

1) IaaS – Infrastructure as a Service

- Provides virtual machines, storage, networks
- Provider manages hardware
- Client manages OS & apps

- **Examples:** AWS EC2, Google Compute Engine, RackSpace, DigitalOcean, IBM Cloud, Oracle Cloud

2) PaaS – Platform as a Service

- Provides OS, middleware, runtime, dev tools
- Client deploys apps
- Provider manages full platform
- **Examples:** Google App Engine, Heroku, Azure PaaS, RedHat OpenShift, AWS Elastic Beanstalk

3) SaaS – Software as a Service

- Complete software delivered via browser
- Client only uses the software
- Provider manages everything
- **Examples:** Gmail, Zoom, Google Apps, QuickBooks Online, Salesforce, SQL Azure

Other XaaS

- **CaaS** – Container as a Service (Kubernetes, Docker Swarm)
- **NaaS** – Network as a Service (SDN, NFV, Cloud WAN)
- **SECaaS** – Security as a Service
- IDaaS, EaaS, etc.

13. Benefits of Cloud Computing

- Environment friendly
- Easy software integration
- Cost efficient
- More secure
- Flexible
- Infinite storage
- Rapid development

- Backup & recovery
- Document control
- Fewer maintenance issues

14. Limitations of Cloud Computing

- Doesn’t work well with low internet speeds
- Outsourcing data increases risk
- Vendor stability issues
- Downtime & accessibility problems
- Data security challenges
- Copyright & piracy risks
- Requires constant internet
- Possible unreliable ICT/power
- Vendor lock-in risk